

PRM

Théorie : Leçon I I

Java :

Les caractères (char) et
les chaînes de caractères (String)

Les caractères (le type primitif `char`)

En Java, le type primitif `char` permet de représenter des caractères. Un caractère littéral est délimité par des apostrophes (*simple quotes*) :

```
char lettre = 'p';
char espace = ' ';
```

Certains caractères n'ont pas de représentation graphique :

```
char retourLigne = '\n';
char tabulation = '\t';
```

Comme les apostrophes délimitent les caractères littéraux et que `\` (*backslash*) permet de noter des caractères spéciaux, voici comment représenter ces deux derniers :

```
char apostrophe = '\'';
char backslash = '\\';
```

Dans ce contexte, on appelle `\` le caractère d'échappement.

Les caractères (le type primitif `char`)

En informatique, chaque caractère (`char`) correspond à une valeur numérique, son indice dans une table (ASCII) dont voici un extrait :

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Remarquez que les 26 lettres sont rangées dans l'ordre alphabétique, que les majuscules précèdent les minuscules et que les chiffres sont rangés de 0 à 9.

Opérations et conversions

Comme, en informatique, les caractères sont représentés par des nombres, il est possible de passer d'une représentation à une autre :

`(int)'d'` donne l'indice du caractère 'd' (soit 100) dans la table ASCII

`(char)100` donne le caractère d'indice 100 de la table ASCII (c.à.d. le 'd')

Exemples :

```
char c = (char)((int)'a' + 1);    //c vaut maintenant 'b'
```

```
char n = (char)((int)'0' + 5);    //n vaut maintenant '5'
```

```
if(c >= 'a' && c <= 'z')
```

```
    System.out.println(c + " est une lettre minuscule");
```

```
int i = (int)'e' - (int)'a';        //i vaut 4 (e est la 5ième lettre)
```

```
char cMaj = (char)((int)c + ((int)'A' - (int)'a')); //cMaj vaut 'B'
```

Les chaînes de caractères (le type référence `String`)

Nous en avons déjà parlé : en Java, le type `String` permet de représenter des chaînes de caractères et les chaînes de caractères littérales sont délimitées par des guillemets (*double quotes*).

```
String message = "Hello world !";
```

Un `String` est une **collection indicée de caractères**. Ces indices sont compris entre 0 et `taille - 1`.

Pour lire une chaîne de caractères, l'outil Scanner propose les méthodes `next()` pour lire un mot et `nextLine()` pour lire une ligne entière.

Comme les types `SeqInt` et `Array`, le type `String` est un **type référence**. Au contraire des `Array`, un `String` est **immuable** : il n'est pas possible de le modifier après sa création (**pas de `set`, `add`, ...**).

Opérations sur les chaînes de caractères

```
String message = "Hello world !";
```

La méthode `int length()` donne la taille d'une chaîne:

```
System.out.println(message.length()); //affiche 13
```

La méthode `char charAt(int pos)` renvoie le caractère situé en position donnée:

```
System.out.println(message.charAt(6)); //affiche w
```

L'opérateur `+` permet de concaténer des chaînes, c'est-à-dire **créer une nouvelle chaîne de caractères** formée de ces deux chaînes mises bout à bout (souvenez-vous qu'un String est immuable) :

```
String nom = "Raoul";
```

```
String msg = "Hello " + nom + " !";
```

Attention : `"somme = " + 1 + 2` donne `"somme = 12"`.

`"somme = " + (1 + 2)` donne `"somme = 3"`.

Comparaison de chaînes

En Java, les opérateurs de comparaison (<, >, ...) ne sont pas définis sur les chaînes de caractères.

Pour comparer l'ordre lexicographique (*l'ordre des caractères dans la table ASCII*), la méthode `int compareTo(String other)` est prévue :

`s1.compareTo(s2)` renvoie

0 si s1 est identique à s2

un négatif quelconque si s1 est avant s2

un positif quelconque si s1 est après s2

Exemple :

```

if(s1.compareTo(s2) > 0) // ""s1 > s2""
    System.out.println(s1 + " est après " + s2);
    
```

Comparaison de références

Les **variables de type référence** (Scanner, SeqInt, Array, String, ...) contiennent l'**adresse** (la référence) de l'objet assigné (référéncé).

L'opérateur == appliqué sur ces variables (référénts) compare leur adresse et non le contenu de l'objet référéncé.

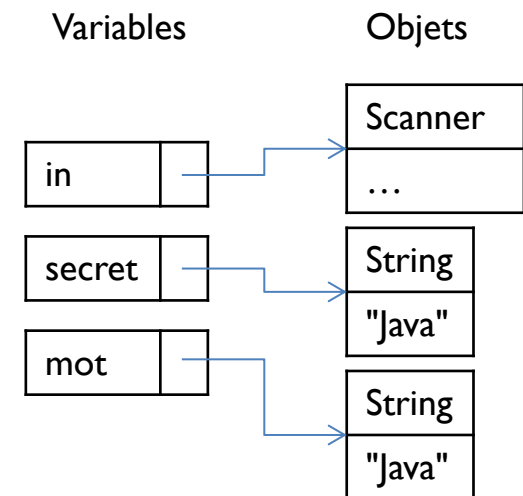
Ainsi, vous ne pouvez pas utiliser l'opérateur == pour déterminer l'identité de deux chaînes : utilisez la méthode compareTo.

```
Scanner in = new Scanner(System.in);
System.out.print("Devinez le mot secret : ");

String secret = "Java";
String mot = in.next();

if(mot == secret)
    System.out.println("Ce message n'est jamais affiché");

if(mot.compareTo(secret) == 0)
    System.out.println("Vous avez trouvé !");
else
    System.out.println("Vous n'avez pas trouvé...");
```



Comparaison de primitifs et de références

```
Scanner in = new Scanner(System.in);

System.out.print("Devinez l'entier secret : ");

int nb = 3;
int val = in.nextInt();
if(nb == val)
    System.out.println("Vous avez trouvé l'entier !");
else
    System.out.println("Vous n'avez pas trouvé...");

System.out.print("Devinez le mot secret : ");

String secret = "Java";
String mot = in.next();

if(mot == secret)
    System.out.println("Ce message n'est jamais affiché");

if(mot.compareTo(secret) == 0)
    System.out.println("Vous avez trouvé !");
else
    System.out.println("Vous n'avez pas trouvé...");
```

Remarquez que les variables de type primitif (int, double, boolean, ...) contiennent leur valeur.

Variables

Objets

